

EJERCICIOS DE REPASO

Programación I — Nivel Introductorio

Funciones · Estructuras de Datos · Manejo de Errores · CSV

Parte 1 — Funciones y Manejo de Errores

Ejercicio 1 — Calculadora de Propinas

Consigna

Crear una función llamada **calcular_propina()** que reciba el monto total de una cuenta de restaurante y el porcentaje de propina deseado.

La función debe:

- Validar que el monto sea mayor a 0
- Validar que el porcentaje esté entre 0 y 100
- Lanzar un **ValueError** con mensaje descriptivo si alguna validación falla
- Retornar el monto de la propina y el total a pagar

Luego, en el programa principal:

- Pedir el monto y el porcentaje al usuario
- Capturar el ValueError con try/except y mostrar el mensaje de error
- Mostrar la propina y el total si los datos son válidos

Ejemplo de uso:

```
monto = float(input('Monto de la cuenta: $'))
porcentaje = float(input('Porcentaje de propina (%): '))
try:
    propina, total = calcular_propina(monto, porcentaje)
    print(f'Propina: ${propina:.2f} | Total: ${total:.2f}')
except ValueError as e:
    print(f'Error: {e}')
```

★ EXTRAS

- Capturar también ValueError al convertir la entrada del usuario a float
- Agregar un tercer parámetro opcional: cantidad de personas para dividir la cuenta
- Usar finally para mostrar siempre un mensaje de despedida

■ CONSEJO TÉCNICO

- `raise ValueError('mensaje')` lanza una excepción manualmente desde dentro de la función
- Un bloque try puede tener múltiples except para distintos tipos de error

Ejercicio 2 — Conversor de Unidades con Menú

Consigna

Crear tres funciones de conversión: `km_a_millas()`, `kg_a_libras()` y `litros_a_galones()`. Cada una recibe un número y retorna el valor convertido.

Cada función debe:

- Validar que el valor no sea negativo
- Lanzar un `ValueError` si el valor es inválido

El programa debe mostrar un menú:

```
1. Kilómetros a Millas (× 0.621371)
2. Kilogramos a Libras (× 2.20462)
3. Litros a Galones (× 0.264172)
4. Salir
```

El menú debe:

- Repetirse con un bucle `while` hasta que el usuario elija Salir
- Capturar `ValueError` si la opción o el valor ingresado no es numérico
- Mostrar 'Opción inválida' sin cortar el programa si se ingresa algo inesperado

★ EXTRAS

- Agregar conversión inversa (millas a km, libras a kg, galones a litros)
- Guardar un historial de conversiones realizadas en la sesión y mostrarlo al salir
- Formatear el resultado con 4 decimales

■ CONSEJO TÉCNICO

- Un `except` sin especificar el tipo captura cualquier excepción — úsalo solo como último recurso
- `except (ValueError, TypeError)`: permite capturar más de un tipo en un solo bloque

Ejercicio 3 — Validador de DNI Argentino

Consigna

Crear una función llamada **validar_dni()** que reciba un string con el DNI del usuario y lo valide siguiendo las reglas del formato argentino.

La función debe validar que:

- El DNI contenga solo dígitos (sin puntos ni espacios)
- Tenga entre 7 y 8 caracteres
- No empiece con 0
- Retornar True si es válido, False si no lo es

Crear también una función formatear_dni() que:

- Reciba un DNI válido como string
- Lo retorne formateado con puntos: XX.XXX.XXX
- Lanzar un ValueError si el DNI no es válido antes de formatear

```
# Ejemplo de salida esperada:
dni = input('Ingresá tu DNI (solo números): ')
try:
    dni_formateado = formatear_dni(dni)
    print(f'DNI válido: {dni_formateado}') # → 32.456.789
except ValueError as e:
    print(f'DNI inválido: {e}')
```

★ EXTRAS

- Limpiar automáticamente puntos y espacios antes de validar con `str.replace()`
- Agregar validación de CUIL: prefijo (20/23/24/27) + DNI + dígito verificador
- Crear una función que valide una lista de DNIs y retorne cuáles son válidos e inválidos

■ CONSEJO TÉCNICO

- `str.isdigit()` retorna True si todos los caracteres del string son dígitos
- `len(string)` permite verificar la longitud del texto ingresado

Parte 2 — Listas, Tuplas, Sets y Diccionarios

Ejercicio 4 — Agenda de Contactos

Consigna

Crear un diccionario que represente una agenda de contactos, donde la clave sea el nombre y el valor sea el número de teléfono.

```
agenda = {  
    "Ana García": "2614-523100",  
    "Luis Romero": "2614-887241",  
    "María Pérez": "2615-334402",  
    "Carlos López": "2614-667890",  
}
```

El programa debe:

- Mostrar todos los contactos ordenados alfabéticamente
- Permitir buscar un contacto por nombre usando dict.get()
- Capturar KeyError si se intenta acceder a una clave que no existe con []
- Mostrar cuántos contactos hay en total

★ EXTRAS

- Agregar y eliminar contactos durante la ejecución
- Buscar por número de teléfono iterando los valores del diccionario
- Detectar si hay teléfonos duplicados usando un set de valores

■ CONSEJO TÉCNICO

- dict.get(clave, valor_default) evita KeyError: retorna el default si la clave no existe
- sorted(diccionario.items()) ordena por clave y retorna lista de tuplas (clave, valor)

Ejercicio 5 — Historial de Ventas

Consigna

Crear una lista de tuplas que represente el historial de ventas de una semana. Cada tupla contiene: (día, producto, cantidad, precio_unitario).

```
ventas = [  
    ("Lunes", "Martillo", 3, 3500.0),  
    ("Lunes", "Taladro", 1, 18500.0),  
    ("Martes", "Destornillador", 5, 1200.0),  
    ("Miércoles", "Sierra", 2, 7200.0),  
    ("Jueves", "Martillo", 4, 3500.0),  
    ("Viernes", "Taladro", 2, 18500.0),  
]
```

El programa debe:

- Calcular el total recaudado (cantidad × precio)
- Mostrar la venta de mayor monto
- Mostrar los productos únicos vendidos usando un set
- Calcular el promedio diario de ventas

★ EXTRAS

- Filtrar todas las ventas de un día específico ingresado por el usuario
- Agrupar ventas por producto y calcular el total por cada uno
- Ordenar la lista de ventas por monto descendente

■ CONSEJO TÉCNICO

- `max(lista, key=lambda x: x[2]*x[3])` encuentra el máximo usando un criterio personalizado
- Las tuplas son inmutables: no se pueden modificar, solo reemplazar en la lista

Ejercicio 6 — Torneos y Participantes

Consigna

Se tienen dos torneos de ajedrez con sus respectivos listas de participantes. Usar operaciones de sets para analizar la participación.

```
torneo_a = {"Ana", "Luis", "Carla", "Pedro", "Sofía", "Tomás"}
torneo_b = {"Luis", "Sofía", "Diego", "Valentina", "Pedro", "Lucía"}
```

El programa debe:

- Mostrar participantes que fueron a ambos torneos (intersección)
- Mostrar participantes que fueron solo al torneo A (diferencia)
- Mostrar participantes que fueron solo al torneo B (diferencia)
- Mostrar el total de participantes únicos entre ambos (unión)

★ EXTRAS

- Agregar un tercer torneo y calcular quién participó en los tres
- Pedir nuevos participantes por teclado y agregarlos al set correspondiente
- Mostrar si los torneos tienen participantes en común antes de hacer la intersección

■ CONSEJO TÉCNICO

- Operadores de set: | (unión), & (intersección), - (diferencia), ^ (diferencia simétrica)
- `set.isdisjoint(otro_set)` retorna True si no tienen ningún elemento en común

Parte 3 — Ejercicio Integrador Completo

Sistema de Gestión de Clientes

Objetivo

Desarrollar un sistema ABM completo de clientes usando funciones, listas de diccionarios, manejo de errores y persistencia automática en CSV.

Estructura de cada cliente

```
{
  "id": "1",
  "nombre": "Ana García",
  "email": "ana@email.com",
  "telefono": "2614-523100",
  "saldo": "15000.00"
}
```

Funcionalidades obligatorias

1. Cargar datos desde CSV al iniciar

- Leer clientes.csv al arrancar el programa
- Si el archivo no existe, comenzar con lista vacía sin lanzar error
- Usar try/except FileNotFoundError para manejar la ausencia del archivo

2. Alta de clientes

- Ingresar todos los datos del cliente
- Validar que el email contenga '@' y '.' — lanzar ValueError si no
- Validar que el saldo sea un número mayor o igual a 0
- Generar el ID automáticamente (máximo ID existente + 1)
- Guardar automáticamente en CSV después de cada alta

3. Mostrar todos los clientes

- Mostrar los datos en formato de tabla prolija
- Capturar el caso de lista vacía e informar al usuario

4. Buscar cliente

- Buscar por ID o por nombre (búsqueda parcial, case-insensitive)
- Informar claramente si no se encontró ningún resultado
- Capturar ValueError si el ID ingresado no es un número entero

5. Modificar saldo

- Buscar el cliente por ID
- Permitir sumar o restar del saldo actual
- Validar que el saldo nunca quede negativo — lanzar ValueError si queda
- Guardar automáticamente después de la modificación

6. Eliminar cliente

- Eliminar por ID con confirmación previa (s/n)
- Capturar ValueError si el ID ingresado no es número
- Guardar automáticamente después de la eliminación

7. Estadísticas

- Cantidad total de clientes
- Cliente con mayor saldo
- Promedio de saldos
- Cantidad de clientes con saldo en 0
- Capturar ZeroDivisionError si no hay clientes al calcular el promedio

Menú esperado

```
1. Agregar cliente
2. Mostrar clientes
3. Buscar cliente
4. Modificar saldo
5. Eliminar cliente
6. Estadísticas
7. Salir
```

Restricciones importantes

- NO debe existir una opción 'Guardar' ni 'Cargar' en el menú
- El guardado debe ser AUTOMÁTICO después de alta, modificación y eliminación
- TODOS los ingresos del usuario deben estar protegidos con try/except
- El programa NO debe cerrarse ante ningún error de usuario

Funciones sugeridas

```
cargar_clientes() guardar_clientes() validar_email()  
agregar_cliente() mostrar_clientes() buscar_cliente()  
modificar_saldo() eliminar_cliente() mostrar_estadisticas()  
menu()
```

★ EXTRAS

- Exportar a un segundo CSV solo los clientes con saldo mayor a cierto umbral
- Validar que no existan emails duplicados al dar de alta
- Agregar campo fecha_alta usando el módulo datetime
- Ordenar el listado por saldo de mayor a menor
- Filtrar clientes por rango de saldo (mínimo y máximo ingresados por el usuario)
- Manejo de errores de escritura en disco con try/except OSError al guardar